PDV WEB · DOCUMENTAÇÃO DE TESTES

Relatório de Defeitos

Campo	Valor
Identificador	RDF-PDV-001 · versão 1.2 · 28/10/2026 (Ciclo 1.2 — banco de dados e Docker)
Escopo	Defeitos do backend `pdv-web-api` encontrados pelos testes, defeitos do sistema desktop legado e inconsistências da documentação
Severidade / prioridade	Definições no Plano de Testes, seção 12

1. Resumo

Origem	Encontrados	Corrigidos e retestados	Corrigidos, reteste pendente	Abertos (aceitos/adiados)
Backend `pdv-web-api` — produto	9	9	0	0
Backend `pdv-web-api` — script de teste	1	1	0	0
Desktop `SistemaVendasEmpresa`	9	0	7	2
Documentação do projeto	5	—	—	4 resolvidos por decisão (ADR-001; DOC-04 virou configuração) · 1 a corrigir no documento
Total de defeitos de software	**19**	**10**	**7**	**2**

Por severidade (somente defeitos de software): Crítica 1 · Alta 3 · Média 10 · Baixa 5. **Defeitos Críticos ou Altos abertos no backend: 0** (critério CS-3 atendido).

2. Defeitos do backend `pdv-web-api`

DEF-WEB-001 — Vendedor recebe 400 em vez de 403 em operações exclusivas do Gerente

Campo	Valor
Severidade / prioridade	Média / P1 (segurança — exposição de informação)
Status	**Fechado** (corrigido, retestado, regressão aprovada)
Módulo	`seguranca/ConfiguracaoSeguranca`
Encontrado por	CT-SEG-005 (matriz RBAC) — 5 de 9 operações falharam na primeira execução
Passos para reproduzir	1. Autenticar como Vendedor. 2. `POST /produtos` com corpo `{}`.
Resultado esperado	`403 acesso-negado`, sem processar o corpo
Resultado obtido	`400 validacao` listando os campos obrigatórios (`nome` , `preco_venda` , ...)

Campo	Valor
Causa raiz	A autorização ficava só no `@PreAuthorize` do controller, avaliado **depois** da leitura e validação (`@Valid`) do corpo. Um usuário sem permissão conseguia mapear as regras de validação de operações às quais não tem acesso.
Correção	Autorização por rota no `SecurityFilterChain` , aplicada antes do processamento MVC. O `@PreAuthorize` permanece como segunda camada.
Reteste	CT-SEG-005 9/9 · `SegurancaIT` 16/16 · `ProdutoControllerWebTest` 12/12 · regressão final 264/264

DEF-WEB-002 — Gerente rebaixado mantinha os privilégios de gerente até o token expirar

Campo	Valor
Severidade / prioridade	**Alta** / P1 (segurança — retenção de privilégio por até 24 h)
Status	**Fechado** (corrigido, retestado, regressão aprovada)
Módulo	`seguranca/ConfiguracaoSeguranca` , `usuario/UsuarioRepository`
Encontrado por	Análise de risco ao escrever a sessão exploratória EXP-02 ("o perfil fica no token até expirar?"). A hipótese foi transformada no caso automatizado CT-SEG-009, que **falhou** na primeira execução.
Passos para reproduzir	1. Promover um vendedor a GERENTE. 2. Ele faz login (token com `perfil = GERENTE`). 3. Outro gerente o rebaixa a VENDEDOR. 4. Com o token antigo, `GET /usuarios` .
Resultado esperado	401 (token invalidado pela mudança de perfil)
Resultado obtido	**200** — o usuário rebaixado continuava listando funcionários e poderia cancelar vendas, dar descontos e fazer sangrias por até 24 h
Causa raiz	O perfil é gravado como claim no JWT e usado para a autorização; o validador conferia só se o usuário estava ativo, não se o perfil ainda era o mesmo.
Correção	O validador do JWT passou a exigir usuário ativo **e** perfil igual ao da emissão (`existsByIdAndAtivoTrueAndPerfil`). Qualquer mudança de perfil invalida os tokens anteriores; o usuário faz login de novo e recebe o perfil atualizado.
Reteste	CT-SEG-009 aprovado · `SegurancaIT` 17/17 · `AutenticacaoIT` 9/9 · `UsuarioIT` 9/9 · regressão final 265/265

Defeitos do Ciclo 1.1 (robustez) — 28/10/2026

Os quatro defeitos abaixo vieram dos **riscos residuais** do Ciclo 1 (Relatório de Execução, seção 7) e do novo teste de carga CT-PERF-005. Cada um ganhou casos automatizados que falhariam sem a correção.

DEF-WEB-003 — Logout e renovação de token se perdiam ao reiniciar a API

Campo	Valor
Severidade / prioridade	Média / P1 (segurança — sessão encerrada volta a valer)
Status	**Fechado** (corrigido, retestado, regressão aprovada)
Módulo	`seguranca/RevogacaoDeTokens`
Encontrado por	Análise de risco do Ciclo 1 (revogação mantida em memória)
Passos para reproduzir	1. Login. 2. Logout. 3. Reiniciar a API (ou atender a requisição por outra instância). 4. Usar o token antigo.
Resultado esperado	401
Resultado obtido	200 até o token expirar (24 h): a lista de tokens revogados ficava num `Map` em memória
Correção	Tabela `tokens_revogados` (migração V3), gravada no logout e na renovação; limpeza agendada dos registros já expirados

Campo	Valor
Reteste	CT-SEG-011 (outra instância da API com o mesmo banco recusa o token), CT-SEG-012 (limpeza), CT-AUTH-005/006 · regressão 313/313

DEF-WEB-004 — Login sem limite de tentativas (força bruta)

Campo	Valor
Severidade / prioridade	Média / P1 (segurança)
Status	**Fechado**
Módulo	`auth/AuthService` , `seguranca/LimitadorDeTentativas`
Encontrado por	Análise de risco do Ciclo 1 (PEND-08)
Resultado obtido	Tentativas ilimitadas de senha por CPF e por IP
Correção	Contagem de falhas no banco (tabela `tentativas_acesso` , atualização atômica): 5 falhas por CPF ou 20 por IP em 15 min bloqueiam por 15 min, com **429 +** `Retry-After` . CPF inexistente é bloqueado da mesma forma (não revela se existe). Bloqueios são auditados (`ACESSO_BLOQUEADO`). Limites configuráveis em `pdv.seguranca.tentativas`
Reteste	CT-SEG-013 a 016, CT-SEG-019 (10 tentativas simultâneas não escapam do limite), CT-AUTH-U05/U06; smoke em servidor real (passo 12)

DEF-WEB-005 — Senha do gerente podia ser adivinhada pela tela de venda

Campo	Valor
Severidade / prioridade	**Alta** / P1 (segurança — um vendedor obteria a senha de um gerente)
Status	**Fechado**
Módulo	`venda/VendaService.autorizarDescontoDireto`
Encontrado por	Revisão de segurança feita junto com a DEF-WEB-004: o campo `senha_autorizacao_gerente` de `POST /vendas` também confere senha
Resultado obtido	Um vendedor podia testar senhas sem limite enviando vendas com desconto; a falha nem era auditada
Correção	Mesmo limitador, por vendedor (5 falhas → 429 por 15 min); cada erro é auditado como `AUTORIZACAO_FALHA`
Reteste	CT-SEG-017, CT-VEN-U24

DEF-WEB-006 — Primeira rajada após a subida passava de 3 s (VAL-07)

Campo	Valor
Severidade / prioridade	Média / P2 (desempenho)
Status	**Fechado**
Módulo	Inicialização da aplicação
Encontrado por	CT-PERF-005 (novo): 20 operadores simultâneos logo após a subida
Resultado obtido	Pior requisição entre **2,8 s e 4,0 s** em 3 execuções; todas as lentas no primeiro ciclo
Causa raiz	Partida a frio: o pool de conexões abria as conexões sob demanda e o código ainda não estava compilado pelo JIT
Correção	`AquecimentoNaInicializacao` : antes de a prontidão (`/actuator/health/readiness`) aceitar tráfego, abre as conexões em paralelo e executa as consultas mais usadas

Campo	Valor
Reteste	3 execuções isoladas: máx. 732 / 744 / 1.115 ms na primeira rajada. Regressão oficial: máx. **284 ms** (primeira rajada) e **981 ms** (regime, 400 requisições)

Defeitos do Ciclo 1.2 (Docker) — 28/10/2026

DEF-WEB-007 e 008 foram encontrados na primeira execução do roteiro de implantação (RT-DEP) no Docker Desktop do Windows, com Git Bash; DEF-WEB-009, por inspeção ao preparar o proxy TLS.

DEF-WEB-007 — Restauração de backup falhava no Git Bash e deixava a API parada

Campo	Valor
Severidade / prioridade	Média / P1 (disponibilidade — recuperação de desastre)
Status	**Fechado**
Módulo	`scripts/restaurar-backup.sh`
Encontrado por	RT-DEP-07
Resultado obtido	`pg_restore: could not open input file "C:/Program Files/Git/backups/…"` ; o banco ficou intacto (transação única), mas a API continuou parada
Causa raiz	O Git Bash converte argumentos que começam com `/` em caminhos do Windows antes de chamar o `docker` ; e o script terminava no erro (`set -e`) antes do `docker compose start api`
Correção	`MSYS_NO_PATHCONV=1` no script e `trap` que religa a API ao sair, com ou sem erro
Reteste	RT-DEP-07 aprovado: vendas 2 → 1 e estoque 8 → 10 após restaurar; API UP e login OK

DEF-WEB-008 — `subir-docker.sh` pedia senha e terminava com erro mesmo com o ambiente saudável

Campo	Valor
Severidade / prioridade	Baixa / P2 (falso alarme na implantação)
Status	**Fechado**
Módulo	`scripts/subir-docker.sh`
Encontrado por	RT-DEP-01
Resultado obtido	Depois de "Pronto", o `psql` pedia senha e falhava: `FATAL: password authentication failed for user "pdv_admin"`
Causa raiz	O banco usa `--auth-local=scram-sha-256` , que exige senha até no socket local do container; o script não a informava
Correção	A consulta roda com `PGPASSWORD` lido das variáveis do próprio container (nenhum segredo passa pela linha de comando do host)
Reteste	RT-DEP-01 aprovado: script termina com código 0 e lista V1–V4

DEF-WEB-009 — Em modo produção (`PDV_EXIGIR_HTTPS=true`) o health check marcaria a API como "unhealthy"

Campo	Valor
Severidade / prioridade	Média / P1 (disponibilidade — o proxy depende de a API estar "healthy" para subir)
Status	**Fechado**
Módulo	`docker-compose.yml` (health check do serviço `api`)
Encontrado por	Inspeção, ao preparar o proxy TLS para RT-DEP-05 (antes de qualquer execução em produção)

Campo	Valor
Resultado esperado sem correção	O health check chama `/actuator/health` por HTTP sem cabeçalho de proxy; com HTTPS obrigatório a API responde 302, a verificação de "UP" falha e o container fica "unhealthy"
Correção	O health check envia `X-Forwarded-Proto: https` (127.0.0.1 é proxy confiável para o Tomcat). A porta 8080 passou a ser publicada só em 127.0.0.1
Reteste	RT-DEP-05: com `PDV_EXIGIR_HTTPS=true` a API fica "healthy" e o proxy sobe

DEF-TST-001 — Cenários BDD de desconto liam "10.00" como 1000 (defeito no script de teste)

Campo	Valor
Severidade / prioridade	Baixa / P2 (não afeta o produto, mas causava falso negativo)
Status	**Fechado**
Encontrado por	BDD-DES R5, R6 e R7 — 3 de 18 cenários falharam: esperado 201, obtido 422 `desconto-invalido`
Análise	A API estava **correta**: recebeu desconto de R$ 1.000,00 e o rejeitou. O tipo `{bigdecimal}` do Cucumber usa o idioma da feature (`# language: pt`), em que o ponto é separador de milhar.
Correção	O passo recebe o valor como texto (`{word}`) e converte com `new BigDecimal(...)` (`PassosPdv`).
Reteste	18/18 cenários, 130/130 passos
Lição registrada	Em features em português, valores decimais devem ser lidos como texto ou com vírgula.

Incidente de ambiente INC-01 (não é defeito)

Na primeira execução do `ProdutoControllerWebTest`, os 12 casos deram **erro** (contexto não inicializou: faltava o bean `HttpSecurity`). No Spring Boot 4, o slice `@WebMvcTest` só configura a segurança com o módulo `spring-boot-starter-security-test`. A dependência foi ajustada no `pom.xml` e os 12 casos passaram. Nenhuma mudança no produto.

---

3. Defeitos do sistema desktop `SistemaVendasEmpresa` (legado)

Encontrados por revisão de código na análise inicial (teste estático). As correções foram aplicadas e o projeto **compila** (`mvnw clean compile`). O reteste funcional depende de PostgreSQL local e da interface gráfica e está roteirizado em 07-roteiro-de-testes-manuais.md (RT-DSK-01 a 07) — **pendente de execução**.

ID	Defeito	Sev.	Prior.	Causa raiz	Correção	Status
DEF-DSK-001	**Estoque pode ficar negativo:** venda acima do saldo era gravada; duas vendas simultâneas podiam vender o mesmo saldo	Crítica	P1	`UPDATE produtos SET estoque = estoque - ?` sem condição; nenhuma validação no carrinho	`UPDATE … WHERE id = ? AND estoque >= ?` , item a item, com rollback se 0 linhas forem afetadas (`VendaRepository`); validação de saldo ao adicionar no carrinho, somando o que já está nele (`VendaController`); `CHECK (estoque >= 0)` no `schema.sql`	Corrigido — reteste RT-DSK-01/02 pendente
DEF-DSK-002	Telas de Clientes, Produtos e Vendedores não abrem fora do Windows ou empacotadas em JAR	Alta	P1	`MainController` carregava `telacliente.fxml` , mas o arquivo é `TelaCliente.fxml` (o classpath diferencia maiúsculas)	Caminhos corrigidos e mensagem clara se o FXML não existir	Corrigido — RT-DSK-03 pendente

ID	Defeito	Sev.	Prior.	Causa raiz	Correção	Status
DEF-DSK-003	`buscarVendasPorVendedor()` sempre devolvia lista vazia	Média	P3	Lia a coluna `comissao`, que não existe no SELECT; a `SQLException` era engolida	Consulta passa a buscar `percentual_comissao`	Corrigido — RT-DSK-04 pendente
DEF-DSK-004	Edição de cliente apagava/ignorava bairro, estado, UF e CEP; a tela não tinha a opção de editar	Média	P2	`UPDATE` com só 6 dos 10 campos; sem handler de atualização	`UPDATE` completo; botão "Atualizar" e carga dos dados ao clicar na tabela	Corrigido — RT-DSK-05 pendente
DEF-DSK-005	`mvn javafx:run` (documentado) não funcionava	Baixa	P3	`pom.xml` sem o `javafx-maven-plugin`	Plugin adicionado; Maven Wrapper incluído	Corrigido — RT-DSK-06 pendente
DEF-DSK-006	Não havia script do banco no repositório	Média	P2	Esquema existia só na máquina do desenvolvedor	`database/schema.sql`, reconstruído a partir das consultas, com restrições (FK, UNIQUE, CHECK)	Corrigido — RT-DSK-07 pendente
DEF-DSK-007	Valores monetários em `double` (erros de arredondamento)	Média	P3	Modelos usam `double`	**Não corrigido no desktop** (refatoração ampla em sistema legado); resolvido no backend web com `BigDecimal` / `NUMERIC(10,2)`	Aceito — adiado
DEF-DSK-008	`ResultSet` não era fechado em `buscarPorDocumento` (vazamento de recurso)	Baixa	P4	Sem try-with-resources	try-with-resources	Corrigido — coberto por RT-DSK-05
DEF-DSK-009	O validador aceita `00000000000` como CPF válido em qualquer cadastro	Baixa	P3	Exceção criada para o Consumidor Padrão vale para todos	**Mantido no desktop** (a tela de vendas depende dele); no backend web o documento é reservado à migração e rejeitado pela API (CT-CLI-007, CT-DOC-005)	Aceito — adiado

4. Inconsistências na documentação do projeto

Não são defeitos de software, mas provocariam defeitos se implementadas literalmente. Todas foram tratadas no ADR-001.

ID	Inconsistência	Documentos	Decisão	Status
DOC-01	Script DDL em sintaxe MySQL; o projeto usa PostgreSQL	Doc. do Projeto §5 × sistema atual	PostgreSQL + Flyway (D-01)	Resolvido
DOC-02	Estoque tripartido × `estoque_atual` único	Doc. do Projeto §1 × API v2.3 / Dicionário	`estoque_atual` + `estoque_minimo`; reserva adiada (D-02)	Resolvido (VAL-08 parcial)
DOC-03	Cronograma com datas de 2027 fora de ordem ("01/10/2027" antes de "01/02/2027")	Plano de Projeto §3.1	Registrado para correção pela equipe (D-05)	Aberto (documento)
DOC-04	Comissão "sobre o valor bruto" × exemplo do contrato calculado sobre o líquido (R$ 10,75 = 5% de R$ 215)	Tabela de Regras 1.1.1.11 × API v2.3 §3.4	Base configurável: `PDV_COMISSAO_BASE=LIQUIDO` (padrão, conforme o contrato) ou `BRUTO` (D-04). Testadas as duas (CT-CALC-009, CT-COM-004)	Resolvido por configuração — a equipe escolhe o valor na implantação

ID	Inconsistência	Documentos	Decisão	Status
DOC-05	Numeração das validações: a Tabela de Planejamento termina em VAL-07 e a Documentação do Projeto vai até VAL-08, com nomes diferentes	Tabela de Planejamento × Doc. do Projeto	Adotada a numeração da Documentação do Projeto (mais recente): VAL-07 = Desempenho, VAL-08 = Reserva de Estoque	Resolvido

5. Lacunas de teste encontradas por teste de mutação

Não são defeitos do produto: são **testes fracos**, que passariam mesmo se o código estivesse errado. Encontradas pelo PIT na primeira rodada (14 mutantes sobreviventes) e eliminadas no mesmo ciclo.

ID	Mutação que sobreviveu	Risco	Reforço aplicado	Status
MUT-01	Remover `produto.baixarEstoque(...)` e `venda.adicionarItem(...)` em `VendaService.registrar`	Venda concluída sem baixar o estoque passaria nos testes unitários	CT-VEN-U19 verifica estoque, itens, resposta e auditoria	Fechado
MUT-02	Remover a auditoria `VENDA_REGISTRO`	Venda sem LOG (Regra 1.2)	CT-VEN-U19	Fechado
MUT-03	`resto < 2` → `resto <= 2` no cálculo do DV de CPF/CNPJ	CPF/CNPJ com DV 9 rejeitado indevidamente	CT-DOC-012 (resto 2 → DV 9)	Fechado
MUT-04	Remover a auditoria da sangria	Sangria sem LOG	CT-CX-U11 verifica o log com saldo anterior/novo	Fechado
MUT-05	`length() < 4` → `<= 4` na máscara de CPF	Máscara errada no limite	CT-AUTH-U04 com exatamente 4 dígitos	Fechado
MUT-06	`Produto.isAtivo()` sempre `true`	Produto inativo vendido	CT-VEN-U16 isola o caso do inativo; CT-VEN-U18 o do inexistente	Fechado
MUT-07	Getters de `Venda` devolvendo `null` e `isCancelada()` sempre `true`	Resposta da venda incompleta	CT-VEN-U19 verifica status, modalidade, data e `isCancelada`	Fechado

Resultado: força dos testes unitários de **91,4%** → **100%** (detalhes no Relatório de Execução, seção 5.3).

5.1 Ciclo 1.1 — lacunas encontradas na rodada 4

As classes alteradas no Ciclo 1.1 (limite de tentativas, base da comissão) e a nova classe avaliada (`InicializadorGerente`) geraram **13 mutantes sobreviventes**. Todos foram eliminados com casos novos ou asserções mais fortes.

ID	Mutação que sobreviveu	Risco	Reforço aplicado	Status
MUT-08	Remover `limitador.verificar(...)` do login e do desconto, e `limitador.limpar(...)` após o acerto	Bloqueio por excesso de tentativas sem efeito	CT-AUTH-U05, CT-VEN-U24; CT-VEN-U13 passa a verificar a limpeza	Fechado
MUT-09	Inverter `bloqueouCpf || bloqueouIp` e a escolha CPF/IP na mensagem	Bloqueio não auditado ou auditado com o alvo errado	CT-AUTH-U06 (tabela de decisão com 4 combinações)	Fechado
MUT-10	Remover `verificarAcesso` do fechamento de caixa e inverter a condição de gerente	Vendedor fecharia o caixa de outro	CT-CX-U16	Fechado
MUT-11	Remover a validação de período das comissões, inverter os limites de data e devolver `null`	Comissão calculada no período errado	CT-VEN-U25	Fechado
MUT-12	Texto do estorno vazio no log de cancelamento	LOG sem os itens estornados (Regra 1.2)	CT-VEN-U20 verifica o texto do estorno	Fechado

ID	Mutação que sobreviveu	Risco	Reforço aplicado	Status
MUT-13	`senha.length() < 8` → `<= 8` no gerente inicial	Senha de 8 caracteres recusada na instalação	CT-USR-U06 (valor-limite)	Fechado

Resultado: força **93,8%** → **99,1%** → **100%** em 12 classes (detalhes no Relatório de Execução, seção 11).